Exercise 3: a second consumer

Your 4 answers, checked, with a picture for each.

#	Your answer	Result
1	No, the exchange already exists, add another queue	**Correct**
2	1 queue for open/close, 1 for audit	**Half right**
3	2 listeners can listen to different keys	**Not right**
4	Yes, the email still goes out	**Correct**

1. Does alert change? No

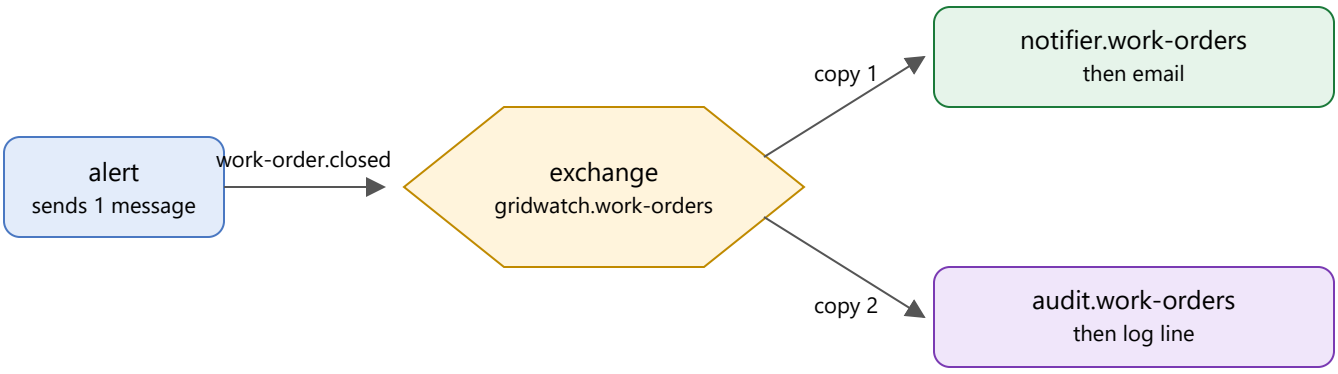
alert only knows the **exchange**. It sends the message there and its job is done. It does not know which queues exist or who reads them. So a new reader is added on the **receiving side**: a new queue plus a binding. alert stays the same.

> Rule: the **sender** owns the exchange. Each **receiver** owns its own queue and binding.

2. How many queues get a copy? 2, per message

Your idea "1 + 1" is close, but open and close do **not** get separate queues. Both go into the same notifier queue, because the binding `work-order.*` matches both keys.

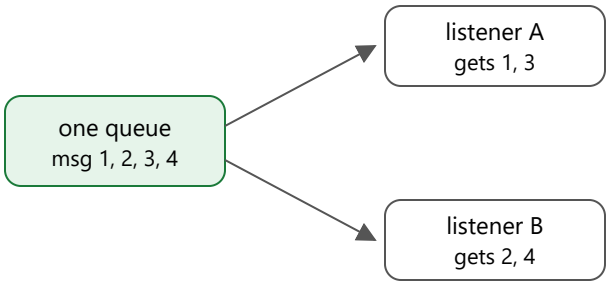
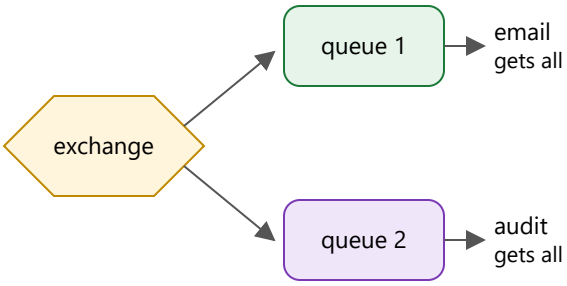
So for **every** message (open or close), the exchange makes **2 copies**: one for the notifier queue, one for the audit queue.



Your task: pick the binding pattern for audit so it also gets **both** open and close.

3. One queue with 2 listeners vs 2 queues These are different

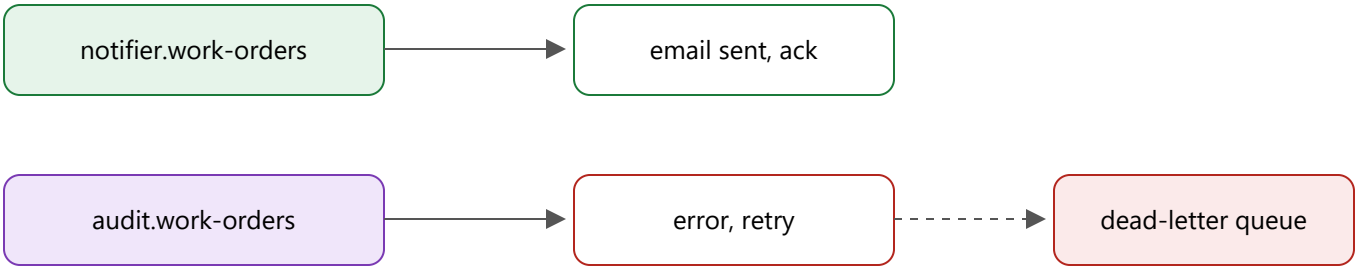
Listeners don't pick routing keys. **Bindings** do that, between the exchange and the queue. A listener just reads whatever lands in its queue.

A. One queue, 2 listeners	B. Two queues, 1 listener each
	
Share the work. Each message goes to only **one** listener. Used to go faster (more workers doing the same job).	**Everyone gets a copy.** Each queue gets **every** message. Used when different jobs need the same event.
Name: *competing consumers*	Name: *publish / subscribe*

> This exercise is **B**. If you put audit as a second listener on the notifier queue, half the messages would be emailed and the other half only logged. That's wrong.

4. Audit fails, does the email still go out? Yes

Each queue has its **own copy** and its **own ack**. If the audit listener throws, only the audit copy is retried and later goes to a dead-letter queue (if you set one up). The notifier copy was already done, and the email was sent.



This is the big win of 2 queues: one consumer failing never blocks another.

Now build it

1. Queue bean `audit.work-orders`, durable, with the name in yaml.
2. Binding bean from that queue to `gridwatch.work-orders`. Pick a pattern that matches both keys.
3. `AuditListener` with `@RabbitListener`, which logs the id, type and time.

4. Trap: with 2 `Queue` beans, Spring doesn't know which one a binding means. Use `@Qualifier` or a matching parameter name.

Done when: one close gives 1 log line, 1 email, and both queues show traffic at localhost:15672.